information

MDPI

Article

Sprint Management in Agile Approach: Progress and Velocity Evaluation Applying Machine Learning

Yadira Jazmín Pérez Castillo [1], Sandra Dinora Orantes Jiménez [1,*] and Patricio Orlando Letelier Torres [2]

[1] Centro de Investigación en Computación, Instituto Politécnico Nacional, Gustavo A. Madero, Mexico City 07738, Mexico; yperezc2022@cic.ipn.mx
[2] Departament de Sistemes Informàtics i Computació, Universitat Politècnica de València, 46022 Valencia, Spain; letelier@dsic.upv.es
* Correspondence: dinora@cic.ipn.mx; Tel.: +52-5729-6000 (ext. 56523)

Abstract: Nowadays, technology plays a fundamental role in data collection and analysis, which are essential for decision-making in various fields. Agile methodologies have transformed project management by focusing on continuous delivery and adaptation to change. In multiple project management, assessing the progress and pace of work in Sprints is particularly important. In this work, a data model was developed to evaluate the progress and pace of work, based on the visual interpretation of numerical data from certain graphs that allow tracking, such as the Burndown chart. Additionally, experiments with machine learning algorithms were carried out to validate the effectiveness and potential improvements facilitated by this dataset development.

Keywords: sprint performance evaluation; progress monitoring; agile project management; machine learning

check for updates

Citation: Pérez Castillo, Y.J.; Orantes Jiménez, S.D.; Letelier Torres, P.O. Sprint Management in Agile Approach: Progress and Velocity Evaluation Applying Machine Learning. *Information* **2024**, *15*, 726. https://doi.org/10.3390/info15110726

Academic Editor: Wenjie Zuo

Received: 25 September 2024
Revised: 22 October 2024
Accepted: 3 November 2024
Published: 12 November 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In today's tech-driven era, data collection, analysis, and application have taken on a pivotal role in supporting informed decision-making across a wide array of fields. The agile approach to work management [1] has been a milestone in transforming how teams address work, prioritizing continuous delivery and adaptation to changes. In recent research, such as the survey conducted in the 16th State of Agile Report titled "Agile adoption accelerates across the enterprise" [2], the current success of the agile approach in the industry is emphasized.

One key difference between agile and traditional methodologies lies in their approach to management and estimation. Conventional models, like the waterfall approach, follow a linear progression with fixed plans and estimations made early in the project lifecycle. This rigidity often hinders flexibility when responding to changes. In contrast, agile methodologies prioritize iterative development, frequent feedback, and the ability to adapt continuously, making estimation more dynamic.

An essential concept in the agile approach is the Sprint, a defined time interval during which new features are developed and integrated into a new product increment [3]. Within this context, assessing the progress and pace of work during a Sprint is essential, especially when working on multiple projects simultaneously. This study evaluates the progress and pace of work during a Sprint to provide an objective and quantifiable perspective, automating the process based on a data model.

It is important to highlight that the approach presented in this work also aligns with the PMBOK guidelines [4], which advocate for data-driven decision-making and continuous adaptation for project success. Integrating agile methodology with Machine Learning (ML) application in Sprint evaluation further reinforces this data-centric approach and constant improvement.

The organization of this work is as follows: Section 2 provides a brief review of the state of the art regarding the evaluation of progress and pace of work supported by ML, including verifying the existence of relevant datasets. Section 3 addresses the research problem, while Section 4 explains the process of creating and designing the dataset and the experimentation phase with ML algorithms. Section 5 discusses the obtained results. Sections 6 and 7 present the conclusions drawn and the avenues for future work.

2. Related Works

In the current project management and software development context, the agile approach has emerged as one of the most popular methods. The evaluation of progress and work pace during a Sprint is essential for making informed decisions and promoting continuous improvement in agile development, ensuring team efficiency and customer satisfaction, as highlighted in the Scrum guide [5]. In this regard, applying ML techniques to analyze Sprint-related data represents an opportunity to offer an objective and quantitative perspective to enhance decision-making.

The existing literature emphasizes the importance of measuring and evaluating work progress in agile development environments [6]. Traditional methods for this purpose, such as formal reviews, have proven insufficient to capture project complexity and its ever-changing nature.

The need to adapt metrics to the agile context has led to research aimed at identifying more relevant indicators associated with a project, such as teamwork pace, costs, and risks, among others, as highlighted in some studies [7,8].

However, at the intersection of agile development and ML, a new horizon of opportunities arises for improving work management, as proposed in [9]. Recent studies have explored the ML algorithm's performance for leveraging data generated during project development to predict progress, quality, and potential obstacles [10–12]. These studies have focused on specific metrics, such as story points or the effort involved in individual sprints.

Additionally, research that has used agile project data as input for ML models has emphasized features such as User Stories (US), prioritization, story points, and US status [13–18]. The data, focused on User Stories (US), offer a variety of features that can be extracted to enrich the analysis with ML. Some of these include prioritization, size or estimation (story points), and the current state of the US. This selection and application of features contribute to a detailed understanding of each US and provide a solid foundation for implementation with ML techniques. All these studies reaffirm that predictive models based on ML can help teams anticipate problems and make informed decisions to adjust their approach in real-time.

In summary, the state of the art at the intersection of agile project management and ML reveals a significant evolution in managing and evaluating projects in agile environments. Although the agile approach has redefined how teams approach software development, accurate assessment of progress and work pace in sprints, especially in multi-project environments, remains a challenge. However, the synergy between agile projects and ML offers a promising perspective for improving team efficiency and productivity and fostering continuous improvement in project management. Developing specific datasets and their application with ML techniques provides a valuable tool for effective agile project management and achieving more excellent value delivery to the customer.

3. Definition of the Problem

The agile development paradigm, focused on adaptability and iterative delivery, has transformed the industry's quest for greater efficiency and customer satisfaction, as demonstrated by company surveys [2]. However, one of the key challenges that has emerged is accurately evaluating and measuring progress and pace during a Sprint.

Traditional measurement methods, such as tracking the completion of tasks (User Stories), while useful for individual sprints, fail to detect anomalies or potential issues that

could affect overall progress. This challenge becomes significantly more pronounced in multi-project environments, where a manager must supervise multiple sprints running in parallel. In these scenarios, it becomes difficult to determine which sprints are on track and which may require immediate attention, especially since most available tools only provide status updates without highlighting those projects or sprints that might be at risk.

A sophisticated approach is required for the effective management of pace and progress across multiple simultaneous projects, one that goes beyond simple quantitative metrics. The challenge lies in developing a comprehensive dataset that includes a range of variables capable of capturing the whole complexity of an agile workflow. These metrics may include factors such as potential work stoppages, excessive increases or decreases in workload, and other unplanned impediments that could affect the sprint's success and the percentage of work completed.

This dataset will be the foundation for applying ML techniques to evaluate sprints across all projects. By leveraging ML, it is possible to automate the detection of sprints that may require immediate intervention, allowing managers to focus on critical areas without manually reviewing each project's progress in detail.

The core problem is that, in multi-project environments, current tools and methods are insufficient for guiding managers on where they should focus their efforts. Currently, tools like Jira provide status indicators, but it is up to the manager to manually analyze each indicator. This gap can be addressed by integrating advanced metrics and ML-based analysis, significantly improving decision-making in agile environments with multiple simultaneous sprints.

4. Development

Currently, several platforms allow project managers to oversee the progress of a project, both in agile and traditional methodologies. In the case of the agile approach, there are tools such as Jira [19], version 9.17.1, software that helps teams manage the software delivery lifecycle, regardless of the agile methodology they choose.

Another relevant tool is Worki, part of the TUNE-UP Process [20]. Its purpose is to facilitate the agile transformation of work teams. Used in both business and academic settings, it offers a comprehensive solution to improve product and service development and maintenance processes, focusing on agile work management. For instance, in a recent project involving classroom groups working on diverse initiatives, Worki has been integrated to enhance collaboration and track progress effectively. The platform enabled students to create a shared workspace to visualize their workflows, prioritize tasks, and provide real-time updates. Using Worki features, sprint reviews and retrospectives were organized, fostering continuous improvement and ensuring that each group remained adaptable and aligned with their project objectives. This practical Worki application in an educational context underscored its effectiveness in supporting agile methodologies and optimizing team performance across various disciplines.

The TUNE-UP Process addresses four aspects that an improvement process centered on agile methods should cover:

- Training in the necessary knowledge (Kanban, Lean Development, Scrum, and Extreme Programming).
- Diagnosis of the area where the agile transformation will take place.
- Supporting tools for the approach. It includes proprietary tools: Agilev-Roadmap [20] for diagnosing and supporting the management of an agile roadmap and Worki, a tool for agile work management.

Considering the variety of existing tools, the Worki tool, Version 3.1.10, was selected for its ease in obtaining real data from repositories related to project data, specifically data from software engineering course projects where students develop projects. With Worki, approximately 200 projects are currently available for analysis, with around three sprints per project.

4.1. Sprint Evaluation in Worki

The Worki tool provides a set of functionalities to track Sprints in a given project. Worki offers a Kanban board and a specific dashboard for Sprints, which includes various charts, such as the Burndown chart, designed to track the Sprint in question. In Worki, the overall evaluation of a Sprint considers the following aspects:

1. Backlog Management and Product Structure: The project backlog management and the assessment of how the product is structured in the current Sprint.
2. Project and Sprint Scope Management: An Analysis is conducted on how the project scope has been managed about the Sprint and whether the established objectives have been met.
3. Time Invested, Estimates, and Re-estimations: Consideration is given to the time spent on the Sprint, initial estimates, and any re-estimations made during development.
4. Progress, Work Pace, and Sprint Completion: Evaluating progress and work pace provides a clear view of Sprint's status and helps identify potential issues. Completing Sprint's work is a crucial objective.
5. Preparation for the Next Sprint: Observation is made on how the next Sprint is being prepared and whether lessons learned are being applied.
6. Product Owner Satisfaction: The satisfaction of the Product Owner is an essential indicator of Sprint's quality and success.

Worki houses a variety of valuable information for project managers. However, only the Progress, Work Pace, and Sprint Completion will be considered for the creation of the proposed dataset. These aspects allow for understanding the status of the Sprint and detecting them in projects that might have inadequate work progress.

To evaluate progress and work pace in Worki, the following aspects are considered:

1. Burndown Chart: The goal is to reduce the remaining effort continuously.
2. Completed and Not Completed Work Chart: This shows the percentage of work completed and not completed. This chart expects to show an increasing and continuous amount of completed work.
3. Maximum Time in Activity Chart: Observes unattended work units in an activity for many days.

In relation to the analysis performed, the focus was initially on the Burndown charts and the completed and not completed work charts, as they most significantly determine progress and work pace.

4.2. Research Questions

Following the evaluation of the Sprint in Worki, the main research questions guiding this study are:

1. How can ML techniques improve the accuracy and reliability of Sprint progress and work pace evaluations?
2. What indicators, extracted from the Worki tool, most effectively contribute to predicting potential issues or delays in Sprints?
3. Can ML models significantly improve detection Sprints that are falling behind compared to traditional evaluation methods?
4. What are the best practices for integrating ML models into agile project management tools like Worki to enhance decision-making?

4.3. Dataset Design

The process for designing the dataset intended to evaluate progress and work pace in Sprints is described in this section. This step is crucial for developing a robust and representative dataset capable of providing accurate and valuable information for analysis.

The design of the data model involved identifying the relevant features recorded during the development of the Sprint, as well as establishing the structure and format for storing these data. An expert was consulted to assist in determining these features and

labeling them appropriately. As previously mentioned, the Worki tool was used to collect data. For this purpose, two main charts were initially used and considered for extracting the necessary features for creating the dataset.

In Figure 1, the burndown chart monitors the work pace by emphasizing the remaining time (measured in hours). This chart is commonly used to track project Sprints, although the time range may vary. In the figure, the red line represents the burndown chart. At the same time, the blue line shows the burnup chart and the green line shows the evolution of the total estimated time. In this way, extra information complements the burndown chart.

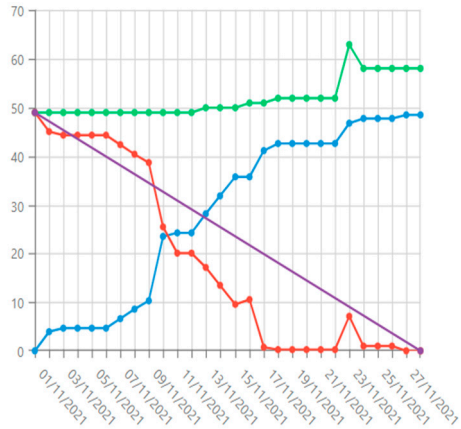

Figure 1. Burndown chart (in hours).

At the beginning of the Sprint, the remaining time is equal to the total estimated time. Subsequently, the remaining time is calculated daily as the difference between the estimated and recorded time.

Worki generates the burndown chart considering only activities marked in the workflows as "estimable." For example, if only the activity "Programming" is marked as "estimable," the daily point on the burndown chart will be calculated as the estimated time minus the recorded time. Focusing on one (or a few) activities simplifies the time estimation and recording process [20].

The burndown chart is complemented using the completed vs. not completed Work chart (in hours). It shows the proportion of completed versus uncompleted work by day based on the total estimated effort (Figure 2). Estimated hours associated with unfinished User Stories (UTs) are shown in orange, while estimated hours associated with completed UTs are shown in green. Only the estimated hours of activities marked as "estimable" in the UT workflows are considered.

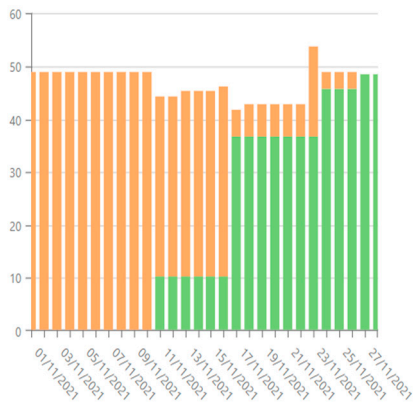

Figure 2. Completed vs. not completed work (in hours).

Understanding the proportion of completed work involves interpreting the burndown chart, as an appropriate reduction rate in the remaining effort should also lead to good

progress in completed work. For example, suppose two-thirds of the Sprint duration has passed, and the burndown chart shows that the estimated time for completed User Stories (UTs) is two-thirds of the total estimated time. In that case, appropriate progress is visible. Nevertheless, if the completed vs. not completed work chart shows that the proportion of completed work is more minor than two-thirds, then the progress is inadequate. The completion of UTs consolidates good progress. An unfinished UT still has the potential to cause rework [20].

4.4. Data Collection and Extraction Process

The next step in designing the dataset was to extract Sprint data using the REST API provided by the Worki platform. Access to the database cannot be provided due to data security, and similarly, access to the API cannot be mentioned in the document. You can visit the TUNE-UP Process [20] website and contact the Worki developers directly.

The SprintId, ProductId, and ProjectId must be provided to access the method that provides the data for the burndown chart. These parameters can be obtained from the API with the appropriate permissions (Figure 3).

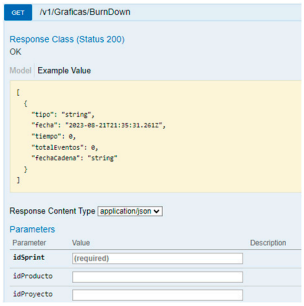

Figure 3. Access to the API method for obtaining the burndown chart.

A JSON object is returned and provides relevant data for the chart, such as time, type, and date. It is necessary to specify the SprintId, ProductId, and ProjectId to access the method that provides data for the completed vs. not completed work chart (see Figure 4). Subsequently, a JSON object is retrieved, and relevant data for the chart are provided, including the number of completed UTs, not completed UTs and the date.

Figure 4. Access to the API Method for the completed vs. not completed UTs chart.

The query of the Worki API methods was conducted through the development of a Python program. It was designed to obtain information related to projects and Sprints and perform calculations and analysis on the collected data. This will shape the proposed dataset, and a CSV file with the results is generated.

The program uses the requests library to make HTTP requests and the CSV library to create the CSV file. Additionally, it utilizes the collections library to keep track of data repetitions in certain operations. These libraries are part of Python. The code is structured in a class called "DataExtraction", which contains several methods for performing specific

tasks, such as data extraction, progress calculations, analysis of the burndown chart, and the completed vs. not completed work chart.

The application is executed with input parameters such as the Worki site URL, headers, and a specific site name, allowing it to adapt its functionality to different environments and projects.

The Python program uses the Worki API to obtain the features that support evaluating the progress and pace of a Sprint. According to this, several key features were previously identified with the help of an expert in the field, based on how progress is visualized in the aforementioned charts. These key features are summarized as follows:

1. Burndown chart data: The program retrieves data from the burndown chart via the Worki API, representing the remaining and recorded work overtime during the Sprint. The program uses these data to calculate Sprint's progress and detect possible excessive reductions or lack of work pace.
2. Percentage of completed work: The program calculates the percentage of completed work based on estimated completed and not completed work units. This provides a more rigorous measure of Sprint progress since each work unit is counted as either completed or not.
3. Work reduction pace: The pace at which remaining work is being reduced in the Sprint is analyzed. The program identifies excessive reduction paces or the lack of significant reduction pace.
4. Progressive increase in work: The program verifies a progressive increase in the remaining effort over time. If the remaining effort is not considerably decreasing, it is considered a sign of a lack of significant progress.
5. Days of work stoppage: The number of days without significant progress in reducing the remaining effort is counted. This feature indicates the number of days the Sprint may have been stalled.

The program processes the data to evaluate the progress and pace of each Sprint, which will be used to categorize the Sprint as Excellent, Good, Fair, or Poor. Additionally, structured data are generated in a CSV file to facilitate subsequent analysis.

In summary, the program uses data from the burndown and completed vs. not completed work charts to obtain features such as percentages of completed work, reduction rates, progressive increase in work, and other features to evaluate Sprint's progress comprehensively. This provides valuable information for decision-making in project management, such as distinguishing between multiple projects to analyze only those presenting anomalies, assisting in Sprint retrospectives, and explaining improvements during or after the Sprint.

4.5. The Final Dataset

The final identified features extracted by the Python program are described in Table 1. For a more detailed review of the program or access to it, please contact the authors of this manuscript.

An example of the extraction results is presented in Table 2, where only the first records are shown. After extraction, 551 sprints were available for analysis. The dataset's features include the following labels: R1 to R11 represent different metrics and attributes relevant to each sprint, as described in Table 1. The label TAG categorizes each sprint based on its performance; it was labeled with the aid of an expert into the aforementioned categories: Excellent (4), Good (3), Fair (2), or Poor (1). It is important to highlight that the key class in this case is the Poor class, as the goal of applying this dataset to ML algorithms is to identify Sprints in this category in the future. If access to the complete dataset is required for analysis and experimentation purposes, please contact the authors of this work.

Table 1. Features to evaluate Sprint progress and work pace.

Key	Name	Description
R1	Percentage of Work Completed	Percentage of work completed by the end of the Sprint.
R2	No Significant Progress?	Are there periods when there are no new completed work?
R3	Times Without Significant Progress	Number of periods without new completed work.
R4	Percentage of Work Stalled	% of days with unfinished work relative to the total days of the Sprint.
R5	Total Days Without Progress	Total days with unfinished work.
R6	No Significant Reduction Pace?	There was no significant reduction in the remaining programming effort between two consecutive days.
R7	Count of Days Without Significant Reduction	Number of days without significant reduction in the remaining programming effort.
R8	Excessive Reduction?	Excessive reduction in the remaining programming effort between two days.
R9	Count of Excessive Reductions	Several excessive reductions in the remaining programming effort between two days. Causes: overestimating task units (fewer programming hours needed) or scope reduction during the Sprint.
R10	Significant Increase?	Increase the remaining programming effort between two days.
R11	Count of Significant Increases	Several significant increases were in the remaining programming effort between the two days. Causes: increase in estimation or introduction of new task units during the Sprint.

Table 2. Example of a final dataset with labeling.

R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	TAG
100	1	1	33.33	7	1	1	0	0	0	0	2
100	1	1	47.62	10	0	0	0	0	0	0	2
65.84	1	1	66.67	14	1	2	1	1	0	0	1

4.6. Data Analysis Methods

After extracting data from Worki and determining the base features for Sprint evaluation with the expert (to shape the dataset), a Python program was implemented to experiment with ML algorithms. The goal was to find the optimal algorithm for solving the problem related to Sprint evaluation. The program includes importing necessary libraries such as pandas, numpy, and various scikit-learn modules for ML tasks. It consists of a class called AgileTrackingML, which is initialized with a CSV file corresponding to the previously developed dataset and some parameters for cross-validation.

The developed class includes methods for splitting the data, performing cross-validation, and implementing tests for seven ML models: K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Multi-Layer Perceptron (MLP), Random Forest (RF), Naive Bayes (NB), Logistic Regression (LR), and AdaBoost (AB). Performance metrics such as Recall, Specificity, F1 Score, and Precision are calculated for each model. Cross-validation was employed to ensure reliable results and prevent overfitting. Additionally, a new data point was predicted using the trained classifier. Overall, the program provided a structured and efficient framework for identifying the best solution to assess the work pace in Sprints. The results from this experiment are shown in Table 3.

The performance result of the models (KNN, SVM, MLP, etc.) demonstrates their ability to assess the work pace in Sprints. Each metric provides key insights into the model's effectiveness, which is crucial when relating these metrics to the original Sprint data and the features used for evaluation.

1. Precision: This metric reflects the proportion of correct, optimistic predictions. In Sprint evaluation, high precision (as seen in the SVM, LR, and RF models) indicates that the model effectively identifies performance categories while avoiding false positives. For example, if a sprint is classified as Excellent, high precision suggests that the model accurately predicts this Sprint category.

2. Recall: Recall measures the model's ability to identify all positive instances correctly. High recall values, as exhibited by the SVM and LR models, are essential for identifying low-performing Sprints (Poor). This ensures that underperforming Sprints are captured accurately, aiding in better planning and optimizing future Sprints.
3. F1 Score: The F1 score balances precision and recall. A high F1 score, seen in the SVM and LR models, suggests that these models can correctly identify performance categories while minimizing errors (false positives and negatives). This balance is vital for accurately assessing team performance in Sprints without over- or underestimating their effectiveness.
4. Accuracy: Accuracy measures the proportion of correct predictions overall. In this case, the SVM, LR, and RF models, with accuracy close to or above 0.85, indicate a robust classification of Sprint performance categories. This implies that these models can correctly classify most Sprints into their respective categories, helping teams track progress more effectively.

Table 3. Experimentation results.

	Recall	**F1**	**Accuracy**	**Precision**
KNN	0.86	0.86	0.86	0.87
SVM	0.88	0.87	0.88	0.88
MLP	0.86	0.86	0.86	0.87
RF	0.87	0.86	0.87	0.87
NB	0.69	0.66	0.69	0.78
LR	0.87	0.87	0.87	0.88
AB	0.77	0.75	0.77	0.75

The experimental results show that several ML models performed well in evaluating work pace in Sprints. SVM, LR, and RF achieved the highest scores across key metrics (Recall, F1-score, Accuracy, and Precision), with values close to or exceeding 0.85 across all metrics. This suggests that these models consistently achieve strong results in assessing Sprint performance and distinguishing between different evaluation classes.

On the other hand, models like KNN, MLP, and AdaBoost also demonstrated respectable performance but slightly lagged behind the top models. However, the Naive Bayes (NB) model yielded lower results across all metrics, indicating potential challenges in generalizing and capturing the complexity of the data.

In summary, these results highlight the effectiveness of ML approaches for evaluating Sprint's work pace, with models like SVM, LR, and RF showing promising performance for addressing the problem.

For a better interpretation of the results, the confusion matrix for the SVM model (Table 4) provides a detailed understanding of the model's performance in classifying Sprint's work pace into four categories: Poor, Fair, Good, and Excellent. The matrix offers insights into how well the model performs in each category and highlights areas where it excels and struggles.

1. Correct predictions: The matrix diagonal represents the correct predictions, where instances are classified into the proper categories. The model classified 129 instances correctly in the Poor category, accurately identifying low-performance Sprints, which is crucial for Sprint evaluation. The SVM model correctly classified 62 instances for the Fair category, but some misclassifications need attention. The Good and Excellent categories saw high correct predictions, with 158 and 134 instances correctly classified, respectively.
2. Misclassifications (off-diagonal elements): For Fair category, 16 instances labeled as Fair were incorrectly classified as Poor. This indicates the model struggles to distinguish some Fair Sprints from Poor ones, leading to false negatives. Twenty-one instances labeled as Fair were classified as Good, showing some overlap between the performance levels of these categories. For the Good category, five instances

labeled as Fair were misclassified as Good, while four instances labeled Excellent were misclassified as Good. This could indicate challenges in recognizing nuances between high-performing Sprints. For the Excellent category: Only four instances of Excellent were incorrectly classified as Good, indicating strong model performance in identifying top-performing Sprints.

Table 4. Confusion matrix SVM.

	Poor	**Fair**	**Good**	**Excellent**
Poor	129	15	2	0
Fair	16	62	21	1
Good	0	5	158	4
Excellent	0	0	4	134

Finally, the high number of correct predictions in the diagonal of the matrix highlights the SVM model's strong ability to discriminate between the different work performance categories in Sprints. Most notably:

1. The model performs well in correctly classifying Poor, Good, and Excellent Sprints, which is critical for identifying low- and high-performing sprints.
2. The misclassification of some Fair instances, mainly being confused with Poor or Good, suggests that the model struggles slightly with mid-range performance levels, where the distinctions between categories are subtler.

Overall, the SVM model is effective in assessing Sprint's performance and, despite some minor misclassifications, demonstrates an impressive ability to classify work performance categories accurately. Its success in identifying low-performance Sprints (Poor class) is significant for teams seeking to identify and improve areas of weakness.

5. Discussion and Analysis of Results

The dataset obtained through extraction from Worki is now available for application using AI techniques. During the extraction process, one of the most significant challenges arose: translating the visual interpretation of charts, such as the burndown chart, into manageable and finite numerical data. This became particularly challenging due to the variability in the duration of Sprints, which often range from 14 to 21 or 28 days, among other examples.

It is important to highlight that the current lack of similar datasets underscores the significance of the work performed in data extraction with Worki and the need to create a valuable and differentiated resource. The potential for future adaptations and refinements to better align with needs is an encouraging prospect. Additionally, a broader horizon is anticipated where this dataset can be leveraged for future research.

On the other hand, the results obtained in the experimentation phase with different AI algorithms are crucial for evaluating the effectiveness of the created dataset and the selected algorithms. These results provide valuable information about which algorithms perform best in evaluating work paces in sprints, along with the implemented dataset and possible areas for improvement in both the algorithms and the dataset features.

Although the algorithms used in the experimentation have proven effective, there are still opportunities to explore other techniques that might further enhance model performance, such as Natural Language Processing (NLP) or Ensemble Models, which combine multiple predictive models. Additionally, the dataset could benefit from new adaptations or refinements to better capture the complexity of sprint evaluation. However, given the high effectiveness demonstrated by the SVM algorithm, it is considered an excellent candidate to advance to a later phase, where it could be implemented in a tool like Worki without ruling out further experiments that might lead to finding an even more optimal algorithm.

Integrating an agile project management tool like Worki could provide additional benefits by enabling a more dynamic and accurate evaluation of work paces in projects.

Finally, in response to the research questions, the results of this study suggest that ML techniques, notably the SVM algorithm, can significantly improve the accuracy of Sprint progress evaluations by leveraging key indicators such as burndown charts and completion percentages. These techniques allow for more precise identification of potential issues or delays in Sprints, addressing the first and second research questions. Furthermore, the integration of ML into Worki enhances real-time decision-making and demonstrates a clear advantage over traditional evaluation methods, answering the third research question. Lastly, the experimentation phase has highlighted best practices for applying ML models in agile management tools, paving the way for continuous refinements, which addresses the fourth research question.

6. Conclusions

This work represents a substantial advancement in converging two fundamental current disciplines: the agile approach and AI. Creating a dataset aimed explicitly at evaluating progress and work pace in agile project sprints is a significant step toward efficient management supported by data-informed decisions.

In the current environment, where technology and adaptability are crucial, the agile methodology has revolutionized how projects are approached. Continuous delivery and flexibility to adjust along the way are central pillars of this methodology, and the precise evaluation of progress and work pace in each sprint has become crucial to project success.

This study identified concrete challenges, and although there are works that combine AI with the agile approach, many focus on specific areas, such as risk analysis and costs, among others, and do not directly address sprint monitoring. Converting the visual interpretation of charts like the burndown chart into numerical data usable by AI algorithms presented a substantial obstacle; additionally, the variability in sprint durations added an extra layer of complexity.

In this work, a data extraction program from the Worki tool was developed based on the TUNE-UP Process approach [20]. The aim was to consider charts from this tool, such as the burndown chart and the representation of completed and non-completed work, to build a dataset subjected to AI techniques to evaluate work progress in a sprint.

Finally, experiments were also conducted with various AI algorithms to assess the effectiveness of the created dataset. The results provided a deeper understanding of how these algorithms can be applied to evaluate progress and work pace in agile project sprints. These experiments served as an initial validation of the proposed approach and highlighted the capability of M to address specific challenges in agile project management.

This study lays the groundwork for future explorations and applications with the implemented dataset. The synergy between AI and agile project management expands the boundaries of analysis and continuous improvement. Evaluating work progress is essential for making informed decisions and achieving effective project direction.

7. Future Work

Future work in this research includes:

1. Improving the dataset: The idea of integrating additional sections of the Sprint evaluation from Worki is considered to strengthen the proposed dataset. This would involve incorporating other tracking charts available in the Worki tool.
2. Model refinement: Conducting experiments with normalized numerical values, using feature weights and possibly including other supporting charts from Worki for tracking. This refinement aims to enhance the model's performance and accuracy.
3. Development of support tools: Creating tools and plugins that facilitate the implementation and use of the model in various platforms and development environments. For example, integrating the model with the Worki platform could provide detailed explanations of the evaluations made by the AI model.

These future directions aim to expand and perfect the proposed model and explore new avenues that contribute to advancing the evaluation and management of progress in agile projects using AI techniques.

Author Contributions: Conceptualization, P.O.L.T.; Methodology, S.D.O.J. and P.O.L.T.; Software, Y.J.P.C.; Validation, Y.J.P.C. and S.D.O.J.; Formal analysis, P.O.L.T.; Investigation, Y.J.P.C., S.D.O.J. and P.O.L.T.; Data curation, Y.J.P.C.; Writing—review & editing, Y.J.P.C.; Project administration, S.D.O.J. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data supporting the reported results of this study are available on request from the corresponding author. The data are not publicly available due to privacy restrictions associated with the confidential data management protocols of Worki.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Sutherland, J. Manifesto for Agile Software Development. Available online: http://agilemanifesto.org/ (accessed on 8 January 2024).
2. Digital.ai. 16th State of Agile Report: Agile Adoption Accelerates Across the Enterprise. Available online: https://stateofagile.com/ (accessed on 16 December 2023).
3. Project Management Institute. *Agile Practice Guide*; Project Management Institute: Newton Square, PA, USA, 2017.
4. Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 6th ed.; Project Management Institute: Newtown Square, PA, USA, 2017.
5. Schwaber, K.; Sutherland, J. La Guía Scrum. La Guía Definitiva de Scrum: Las Reglas del Juego. 2020. Available online: https://repositorio.uvm.edu.ve/server/api/core/bitstreams/5b4aef9b-52f7-49c5-8875-612b1b1dcbc0/content (accessed on 2 August 2022).
6. Jadhav, A.; Kaur, M.; Akter, F. Evolution of Software Development Effort and Cost Estimation Techniques: Five Decades Study Using Automated Text Mining Approach. *Math. Probl. Eng.* **2022**, *2022*, 5782587. [CrossRef]
7. Shamshurin, I.; Saltz, J.S. A predictive model to identify at-risk Kanban teams. *Model Assist. Stat. Appl.* **2019**, *14*, 321–335. [CrossRef]
8. Sousa, A.; Faria, J.P.; Mendes-Moreira, J.; Gomes, D.; Henriques, P.C.; Graça, R. Applying Machine Learning to Risk Assessment in Software Projects. In *Communications in Computer and Information Science (CCIS)*; Springer: Berlin/Heidelberg, Germany, 2021; Volume 1525, pp. 104–118. [CrossRef]
9. Mashkoor, A.; Menzies, T.; Egyed, A.; Ramler, R. Artificial Intelligence and Software Engineering: Are We Ready? *Computer* **2022**, *55*, 24–28. [CrossRef]
10. Abadeer, M.; Sabetzadeh, M. Machine Learning-based Estimation of Story Points in Agile Development: Industrial Experience and Lessons Learned. In Proceedings of the IEEE 29th International Requirements Engineering Conference Workshops (REW), Notre Dame, IN, USA, 20–24 September 2021; pp. 106–115. [CrossRef]
11. Choetkiertikul, M.; Dam, H.K.; Tran, T.; Pham, T.; Ghose, A.; Menzies, T. A Deep Learning Model for Estimating Story Points. *IEEE Trans. Softw. Eng.* **2019**, *45*, 637–656. [CrossRef]
12. Ramessur, M.A.; Nagowah, S.D. A predictive model to estimate effort in a sprint using machine learning techniques. *Int. J. Inf. Technol.* **2021**, *13*, 1101–1110. [CrossRef]
13. Periyasamy, K.; Chianelli, J. A Project Tracking Tool for Scrum Projects with Machine Learning Support for Cost Estimation. *EPIC Ser. Comput.* **2021**, *76*, 86–94. [CrossRef]
14. Malgonde, O.; Chari, K. An ensemble-based model for predicting agile software development effort. *Empir. Softw. Eng.* **2019**, *24*, 1017–1055. [CrossRef]
15. Zia, Z.; Kamal, T.; Ziauddin, Z. An Effort Estimation Model for Agile Software Development. *Adv. Comput. Sci. Its Appl. (ACSA)* **2012**, *2*, 314–324.
16. Batarseh, F.A.; Gonzalez, A.J. Predicting failures in agile software development through data analytics. *Softw. Qual. J.* **2018**, *26*, 49–66. [CrossRef]
17. Dragicevic, S.; Celar, S.; Turic, M. Bayesian network model for task effort estimation in agile software development. *J. Syst. Softw.* **2017**, *127*, 109–119. [CrossRef]

18. Srivastava, P.; Srivastava, N.; Agarwal, R.; Singh, P. Estimation in Agile Software Development Using Artificial Intelligence. In *Lecture Notes in Networks and Systems*; Springer: Berlin/Heidelberg, Germany, 2022; Volume 376, pp. 83–93. [CrossRef]
19. Atlassian. Jira. Available online: https://www.atlassian.com/es/software/jira (accessed on 8 January 2024).
20. TUNE-UP Process. Available online: http://www.tuneupprocess.com/ (accessed on 8 January 2024).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.